



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: P.V. KARISHMIKA

Course Code: CSA0802

Course Name: PYTHON

Course Faculty: Dr. A.Jegatheesan

Reg.No.: 192511453

Slot: C

Project Title: WASTE MANAGEMENT SYSTEM

Module: DICTIONARY OPERATIONS

Module Photographs : (3 photographs –Module Photo, Individual student contribution module work in the project and presentation image)

Smart Waste Collection and Recycling Management System

Waste Type: food waste
Quantity (kg): 15
Category: wet waste
Recyclable (Yes/No): no

ADD WASTE
DISPLAY WASTE
SEARCH WASTE
UPDATE WASTE
DELETE WASTE
SAVE DATA
LOAD DATA

Success
Waste record deleted successfully
OK

Smart Waste Collection and Recycling Management System

Waste Type: food waste
Quantity (kg): 15
Category: wet waste
Recyclable (Yes/No): no

ADD WASTE
DISPLAY WASTE
SEARCH WASTE
UPDATE WASTE
DELETE WASTE
SAVE DATA
LOAD DATA

Success
Waste record updated successfully
OK

Project Description : (here you write what you did in this project (contribution) including Model Description

Individual Contribution – Module 2: File Handling

The second module of the **Smart Waste Collection and Recycling Management System** focuses on storing and retrieving waste records using Python file handling. The main purpose of this module is to ensure that the waste information stored in the dictionary is not lost when the application is closed. I contributed to the implementation and testing of the Save and Load operations.

I implemented the **Save Data** function using Python file handling. The program opens a file named `waste_data.txt` in write mode. The waste records stored in the dictionary are accessed using the `items()` method. The waste type, quantity, category, and recyclable status are then written into the file. A separator symbol is used between the different values so that the information can be separated correctly when it is loaded later.

I also contributed to the **Load Data** function. This function opens the `waste_data.txt` file in read mode and reads the stored records line by line. Empty lines are ignored, and the `split()` operation is used to separate the values stored using the separator. The extracted values are assigned to the appropriate variables and stored back into the `waste_data` dictionary.

Error handling was included in the Load operation using try and except. If the user tries to load data when the file does not exist, the program catches the `FileNotFoundError` and displays an appropriate error message instead of terminating unexpectedly.

I tested the file handling module by first adding different waste records to the dictionary and using the Save Data option. I checked whether the `waste_data.txt` file was created and whether the records were stored correctly. I then closed and reopened the application and used the Load Data option to verify that the previously saved records could be restored to the dictionary.

I also tested the module with multiple records such as Plastic, Paper, and Food Waste. The loaded records were compared with the original records to ensure that the waste type, quantity, category, and recyclable status were correctly restored.

Through this module, I gained practical knowledge of Python file handling, reading and writing text files, file modes, string splitting, loops, exception handling, and data persistence. This module complements the dictionary-based system by allowing waste records to be stored permanently and retrieved whenever required.

Student Signature

Guide Signature